SYSTEM ANALYSIS DOCUMENTATION (SAD)

CO Forever: TernakAI

1. Introduction

This section introduces "TernakAI," the strategic AI-powered decision intelligence solution for poultry disease prevention, transforming fragmented farm signals into actionable decisions.

1.1 Purpose:

This system analysis document highlights the technical scope and design-related decisions behind the TernakAI development. The key elements of the system covered are:

1. **Architecture:** A lightweight, mobile-first cloud application. The backend separates deterministic data processing (baselines, deviations) from the AI microservice (RAG + Z.AI GLM) to operate efficiently.
2. **Data Flows:** How structured data (temperature, feed, mortality) and unstructured data (farmer notes) flow from a mobile form, through the deterministic engines (Layers 2, 3, 4), and into the Z.AI GLM layer to return prioritized action plans.
3. **Model Process:** The end-to-end workflow starting from daily farm data logging, progressing to risk scoring, disease hypothesis generation, and finally outputting constraint-aware preventive actions.
4. **Role of Reference:** This document acts as a reference for the development team to understand the strict boundary between the deterministic engines and the GLM reasoning layer.

1.2 Background:

Malaysia's poultry industry contributes ~RM13 billion annually. Smallholder farms (1,000–10,000 birds) face a critical decision-making gap regarding disease outbreaks.

1. **Previous Version (Status Quo):** Farmers rely on intuition-based, reactive decisions. Fragmented signals arrive separately with no unified interpretation. By the time symptoms are visible, outbreaks are 48–72 hours advanced, locking in 30–50% mortality and up to RM17,500 in losses for a typical 5,000-bird flock.
2. **New Capabilities:** TernakAI introduces early-warning detection. It integrates Z.AI GLM to convert multi-signal patterns into predictive insights, shifting farm interventions from reactive to preventive, potentially saving RM7,000–14,000 per incident.

1.3 Target Stakeholder:

Stakeholders	Roles	Expectations
Owner	Inputs daily shed metrics and text observations; acts on alerts.	Simple, low-bandwidth mobile interface; clear, actionable, and constraint - aware recommendations in plain language.
Farm Worker	Performs daily manual data entry (feed, mortality) into the mobile app.	A fast, frictionless input form in Malay that does not complicate their daily physical chores.
Veterinarian	Escalation point for Critical (81-100) risk scores.	Accurate symptom history and GLM-generated hypotheses grounded in DVS protocols.
Contract Company	Monitors aggregate risk across multiple contracted smallholder farms.	Early warning of potential flock failures to protect supply chain investments and schedule early harvesting if needed.
Development Team	Builds the deterministic engines, RAG pipeline, and GLM integration.	Clear API contracts, distinct separation between deterministic logic and LLM reasoning.
System Administrator	Maintains the cloud infrastructure (AWS) and monitors API health.	Real-time monitoring dashboards, automated alerts for API failures, and clear rollback protocols.

2. System Architecture & Design

This section introduces "TernakAI," the strategic AI-powered decision intelligence solution for poultry disease prevention, transforming fragmented farm signals into actionable decisions.

2.1 High Level Architecture:

2.1.1 Overview:

Type	Details
System	Web/Mobile Application (React/Next.js or Flutter Web)
Architecture	Client-Server API with local Vector Store (RAG) and external LLM

TernakAI uses a layered backend approach. The user interacts via a mobile dashboard. The backend (FastAPI) handles Layer 2 (Baseline), Layer 3 (Risk Scoring), and Layer 4 (Projection) using deterministic math. The output of these engines is then passed to Layer 5 (The AI Microservice), which triggers the RAG retrieval and Z.AI GLM API.

2.1.2 LLM as Service Layer

Z.AI GLM is the non-removable core reasoning engine that transforms raw risk scores into actionable intelligence (Diagnosis, Action Plans, Scenario Narration).

2.1.3 Dependency Diagram

a. **Prompts:** Constructed dynamically by fusing structured deviations (e.g., "Feed -18%, Temp +3°C") and unstructured farmer notes ("ayam senyap").
b. **Context Window:** Injected with retrieved knowledge from the FAISS/ChromaDB Vector Store (chunked DVS guidelines, MARDI research, WOAH manuals) and farm context (flock age/size).
c. **Parsing:** The backend receives GLM's generated hypothesis, structured action recommendations (Immediate, 6hr, 12hr, 24hr), and confidence scores, formatting them for the frontend UI.
d. **Token Limitations:** Instead of sending raw, minute-by-minute data, only the *calculated deviations* and *risk scores* from the deterministic engines are sent to the GLM, heavily reducing token count.

e. **API Calls:** Frontend calls FastAPI backend -> Backend queries FAISS -> Backend calls Z.AI GLM API -> Backend pushes PostgreSQL data.
f. **Interactions:** The PostgreSQL database stores daily logs and baselines. The RAG pipeline ensures hypotheses are not hallucinated but grounded in stored veterinary PDFs.

2.1.4 Sequence Diagram

a. Farmer submits daily log (Feed, Temp, Mortality, Notes).
b. Backend calculates 3-day rolling averages and deviations.
c. Backend computes composite Risk Score (0-100).
d. If Risk > 30 (Moderate/High/Critical), the backend triggers RAG.
e. FAISS returns matching disease profiles.
f. GLM generates interpretation, hypothesis, and prioritised actions.
g. The system triggers Smart Alert and updates Mobile Dashboard.

2.2 Technological Stack:

2.2.1 Frontend

React (Next.js) or Flutter Web (Mobile-first, low-bandwidth).

2.2.2 Backend

Python (FastAPI) for lightweight, async API and GLM orchestration.

2.2.3 Database

SQLite (Hackathon MVP) / PostgreSQL (Scale). FAISS or ChromaDB for RAG vector storage.

2.2.4 Cloud/Deployment

AWS EC2 / Vercel (Frontend) and AWS RDS.

2.3 Key Data Flows:

2.3.1 Data Flow Diagram (DFD)

a. **Movement:** User Mobile -> HTTP POST -> FastAPI. FastAPI routes data to PostgreSQL (persistence) and Risk Engine. Risk Engine output -> RAG/GLM API -> Formatted JSON Response -> User Dashboard.
b. **Integrity:** Risk weights (Feed 0.35, Temp 0.25, Mortality 0.25, Interaction 0.15) are strictly enforced before AI synthesis.

2.3.2 Normalized Database Scheme

a. Farm -> Flock -> Daily_Logs (1:M). Daily_Logs connects to AI_Assessments to store historical hypotheses and feedback loops for future prompt refinement.

3. Functional Requirements & Scope

3.1 Minimum Viable Product

The Minimum Viable Product (MVP) for TernakAI is strategically scoped to demonstrate our core value proposition within the strict time constraints of the hackathon sprint. By focusing exclusively on a frictionless mobile data ingestion flow fused directly with the Z.AI GLM reasoning engine, we can prove the technical feasibility of shifting poultry management from reactive intuition to proactive, data-driven intervention. This foundational feature set ensures we deliver a fully functional, end-to-end workflow—from a farmer logging daily metrics to receiving prioritized, constraint-aware AI recommendations—without getting bottlenecked by Phase 2 hardware integrations.

#	Features	Description
1	Multi-Signal Input Form	Manual daily capture of shed temperature, feed intake, mortality, and unstructured farmer text notes.
2	Risk Scoring Engine	Deterministic baseline calculator returning a weighted risk score (Low to Critical) and projection matrix.
3	GLM Interpretation & Action	Fuses data to generate a disease hypothesis (RAG-grounded) and provides constraint-aware, time-bound action plans.
4	Smart Dashboard & Alerts	Visual risk trends, colour-coded alerts (green / yellow / orange / red), and plain-language scenario narrations.

3.2 Non-Functional Requirements (NFRs)

While the functional features define what TernakAI does, our Non-Functional Requirements (NFRs) dictate how reliably and inclusively it operates in the real world. Given our primary target demographic of Malaysian smallholder poultry farmers, the system must be rigorously engineered to overcome common rural constraints, such as intermittent, low-bandwidth internet connectivity and varying levels of digital literacy. Consequently, our architectural priorities heavily emphasize mobile accessibility, strict cost-efficiency regarding LLM token execution, and transparent traceability to build and maintain user trust in the AI's diagnostic reasoning.

Quality	Requirements	Implementation
Accessibility	The system must work for smallholders in rural areas with poor internet connectivity.	Mobile-first web app optimized for low-bandwidth; entirely text/data-based (no heavy assets).
Traceability	Hypotheses must not be hallucinated; farmers need to trust the AI's diagnosis.	Implementation of RAG. All GLM outputs must strictly append a source citation (e.g., DVS Section 4.2).
Token Latency	GLM analysis must return quickly to maintain user engagement during data logging.	Pre-process data deterministically. Async API calls with frontend loading skeletons.
Cost Efficiency	Avoid unnecessary LLM API calls for normal/safe days.	GLM is only triggered if the deterministic Risk Score crosses the threshold from "Low" to "Moderate" (>30).
Data Resilience	Farm data logs must not be lost if the user loses network connectivity while inside the poultry shed.	Utilize local storage buffering on the mobile app to temporarily cache form inputs, automatically syncing to the backend once the connection is restored.

3.3 Out of Scope / Future Enhancements

a. Bilingual Conversational Interface (Chatbot for follow-up questions).
b. IoT Sensor Integration (Automated Phase 2 hardware integration).
c. Automated deployment of ventilation systems via edge-device APIs.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

4.1.1 Grayscale Rollout

Testing will be simulated using a 3-Day Outbreak Scenario (e.g., CRD progression). A/B testing of prompt structures will be conducted to see which phrasing drives the most immediate farmer comprehension.

4.1.2 Emergency Rollback (ER) / Graceful Degradation

If the Z.AI GLM API is down, the system gracefully degrades to showing only the Deterministic Risk Score, Layer 4 Projections, and raw deviations, ensuring the farmer still gets basic alerts.

4.1.3 Priority Matrix

P1 Critical: RAG retrieval returns conflicting data or GLM rate limit exceeded. Action: Trigger cache of generalized DVS protocols based solely on the composite risk score.

4.2 Monitoring

4.2.1 System Health & Damage Triggers

The planning for the monitoring plan involves continuous tracking of the FastAPI backend and PostgreSQL database using AWS CloudWatch to monitor system health. Key metrics include CPU utilization, database connection pools, and HTTP 5xx error rates. If any damage to the health of the system is detected (e.g., the API error rate exceeds 2% over a rolling 5-minute window), it will immediately trigger an automated PagerDuty alert to the System Administrator.

4.2.2 GLM API Latency

The system monitors the response time and token consumption of the Z.AI GLM API. If the API latency exceeds the p95 threshold of 4 seconds or hits the rate limit cap, the system automatically

triggers the graceful degradation state to ensure the mobile app remains responsive and user data is not lost.

4.2.3 Algorithmic Feedback Loop

We continuously monitor the delta between the AI's projected mortality/disease hypotheses and the actual farmer-confirmed outcomes. If a specific region or flock age consistently shows a deviation of >15% from the deterministic baseline, the data science team is alerted to manually review and recalibrate the Layers 2 & 3 risk weights.

4.3 Assumptions

a. Farmers will consistently log data at least once every 24 hours.
b. Market value of birds (~RM7/bird) and mortality profiles align with standard DVS historical outbreak data.

4.4 External Dependencies

Tools	Purpose	Risks
Z.AI GLM API	Core engine for hypothesis generation, reasoning, and context-aware action planning.	High - If the API fails, the system reverts to raw, un-synthesized data (losing its primary value proposition).
FAISS / ChromaDB	Local vector store for the veterinary knowledge base.	Low - Runs locally/within the backend container, high stability.

5. Project Management & Team Contributions

5.1 Project Timeline (3-Day Sprint)

Day 1: Architecture setup, PostgreSQL schema design, FAISS Vector Store initialization with curated DVS PDFs, and development of Deterministic Engines (Layers 2, 3, 4) in Python.

Day 2: Z.AI GLM prompt engineering, RAG integration, backend-to-frontend API connection, and core Frontend Dashboard development.

Day 3: End-to-end system integration, execution of the "Simulated 3-Day Outbreak Scenario" for live demo preparation, UI/UX polishing, and bug fixing.

5.2 Team Members Role

Member	Role	Key Output
Chew Kean Hong	Data & Deterministic Engine	JSON API: risk score, deviations, projections
Choong Zhuo Lin	GLM Integration & Reasoning	GLM prompt chain, interpretation, recommendations
Soo Kang Shi	RAG & Knowledge Base	Retrieval pipeline with citations
Leow Shen En	Frontend & Dashboard	Mobile-first UI, input form, dashboard
Lee Wei Yin	Demo, Pitch & Integration	End-to-end demo, pitch deck, testing

5.3 Recommendations

To handle context windows effectively during the hackathon, ensure the DVS and MARDI PDFs are aggressively cleaned and chunked (e.g., 500-token blocks) before embedding to maximize the relevance of the RAG context injected into the GLM.